

```
In [1]: 1 import nltk
        2
        3 nltk.download('punkt')
        4 nltk.download('stopwords')
        5 nltk.download('wordnet')
        6 nltk.download('averaged_perceptron_tagger')
        7
```

```
[nltk_data] Downloading package punkt to
[nltk_data]   C:\Users\Welcome\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]   C:\Users\Welcome\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]   C:\Users\Welcome\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]   C:\Users\Welcome\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]   date!
```

Out[1]: True

```
In [2]: 1 from nltk.stem import PorterStemmer
        2 ps = PorterStemmer()
```

```
In [3]: 1 e_words = ["wait", "waiting", "waited", "waits"]
```

```
In [9]: 1 for i in e_words:
        2     rootWord = ps.stem(i)
        3     print(f"{i} -> {rootWord}")
```

```
wait -> wait
waiting -> wait
waited -> wait
waits -> wait
```

```
In [11]: 1 from nltk.stem import WordNetLemmatizer
        2 wl = WordNetLemmatizer()
        3 from nltk.tokenize import word_tokenize
```

```
In [12]: 1 text = "studies studying cries cry"
```

```
In [14]: 1 tokenized_words = word_tokenize(text)
        2 print("Tokenized words:", tokenized_words)
```

Tokenized words: ['studies', 'studying', 'cries', 'cry']

```
In [15]: 1 for i in tokenized_words:
2         lemma = wl.lemmatize(i)
3         print(f"Lemma for {i}: {lemma}")
```

```
Lemma for studies: study
Lemma for studying: studying
Lemma for cries: cry
Lemma for cry: cry
```

```
In [17]: 1 data = "The pink sweater fit her perfectly"
2         words = word_tokenize(data)
```

```
In [18]: 1 for word in words:
2         print(nltk.pos_tag([word]))
```

```
[('The', 'DT')]
[('pink', 'NN')]
[('sweater', 'NN')]
[('fit', 'NN')]
[('her', 'PRP$')]
[('perfectly', 'RB')]
```

```
1 #Algorithm for Create representation of document by calculating TFIDF
```

```
In [19]: 1 import pandas as pd
2         import math
```

```
In [21]: 1 documentA = 'Jupiter is the largest Planet'
2         documentB = 'Mars is the fourth planet from the Sun'
3         xA = documentA.split(' ')
4         xB = documentB.split(' ')
```

```
In [24]: 1 uniqueWords = set(xA).union(set(xB))
2         print(uniqueWords)
```

```
{'is', 'largest', 'from', 'Jupiter', 'fourth', 'Sun', 'planet', 'the', 'Ma
rs', 'Planet'}
```

```
In [25]: 1 numOfWordsA = dict.fromkeys(uniqueWords, 0)
2         for word in xA:
3             numOfWordsA[word] += 1
```

```
In [26]: 1 numOfWordsB = dict.fromkeys(uniqueWords, 0)
2         for word in xB:
3             numOfWordsB[word] += 1
```

```
In [27]: 1 def computeTF(wordDict, bagOfWords):
2         tfDict = {}
3         bagOfWordsCount = len(bagOfWords)
4         for word, count in wordDict.items():
5             tfDict[word] = count / float(bagOfWordsCount)
6         return tfDict
7
8 tfA = computeTF(numOfWordsA, xA)
9 tfB = computeTF(numOfWordsB, xB)
```

```
In [28]: 1 def computeIDF(documents):
2         N = len(documents)
3         idfDict = dict.fromkeys(documents[0].keys(), 0)
4         for document in documents:
5             for word, val in document.items():
6                 if val > 0:
7                     idfDict[word] += 1
8         for word, val in idfDict.items():
9             idfDict[word] = math.log(N / float(val)) if val > 0 else 0
10        return idfDict
11
12 idfs = computeIDF([numOfWordsA, numOfWordsB])
```

```
In [29]: 1 def computeTFIDF(tfBagOfWords, idfs):
2         tfidf = {}
3         for word, val in tfBagOfWords.items():
4             tfidf[word] = val * idfs[word]
5         return tfidf
6
```

```
In [30]: 1 tfidfA = computeTFIDF(tfA, idfs)
2         tfidfB = computeTFIDF(tfB, idfs)
```

```
In [31]: 1 df = pd.DataFrame([tfidfA, tfidfB], index=['Document A', 'Document B'])
2         print(df)
```

```

           is  largest      from  Jupiter  fourth      Sun  plane
t \
Document A  0.0  0.138629  0.000000  0.138629  0.000000  0.000000  0.00000
0
Document B  0.0  0.000000  0.086643  0.000000  0.086643  0.086643  0.08664
3

           the      Mars  Planet
Document A  0.0  0.000000  0.138629
Document B  0.0  0.086643  0.000000
```

```
In [ ]: 1          NAME: Sahil Narke
2          ROLL NO:13245
```

Type your text

